



Figure 1: Movies



Figure 2: Example

tutorials too for some basic hands-on experience, like the movie example shown in Figure 1.

The next generation query language is Cypher, which is used in Neo4j. Cypher's syntax provides a familiar way to match patterns of nodes and relationships in the graph. Cypher has the concept of nodes, relationships and labels, which in layman terms can be related to rows, joins and tables, respectively, in the RDBMS. While creating a node, we give an identifier in braces like (P) for the person. All the nodes can be assigned roles or types called labels. We can establish a relationship between nodes and name them by specifying the name in big braces like [child].

A simple example of a Cypher query is given below:

```
CREATE (Bajrangibhaijaan:Movie {title:'Bajranghi Bhaijaan',
released:1999, tagline:'Selfie Le Re'})
CREATE (KabirKhan:Person (name:'Kabir Khan', born:1967))
CREATE (SalmanKhan:Person (name:'Salman Khan', born:1964))
CREATE (SalmanKhan)-[:ACTED_IN {roles:['Bajranghi Prasad']}]
->(Bajrangibhaijaan)
(KabirKhan)-[:DIRECTED]->(Bajrangibhaijaan)
```

```
import java.io.FileInputStream;

public class DBConnectionManager {

    private static Connection connection;
    private static Statement stmt;
    private static Properties properties = new Properties();
    public static Connection getConnection(){
        if(connection == null)
        {
            try {
                Class.forName("org.neo4j.jdbc.Driver");
                connection = DriverManager.getConnection(
                    "jdbc:neo4j://localhost:7474","neo4j","1qB5");
            } catch(Exception e) {
                e.printStackTrace();
            }
        }
        return connection;
    }

    public static Statement getStatement() {
        // TODO: Auto-generated method stub
        if(stmt == null)
        {
            try {
                stmt = getConnection().createStatement();
            } catch (SQLException e) {
                // TODO: Auto-generated catch block
                e.printStackTrace();
            }
        }
        return stmt;
    }
}
```

Figure 3: DBConnectionManager.

```
WITH SalmanKhan as a
MATCH (a)-[:ACTED_IN]->(m)<-[:DIRECTED]-(d) RETURN a,m,d
```

In this Cypher query, we are creating three nodes with two labels, *Movie* and *Person*. Thereafter, we are making relations between the nodes as 'Acted in' and 'Directed'. In the last statement, I have displayed the result using the 'Return' command. The graph is shown in Figure 2.

Developing applications using Neo4j is fun as it comes with drivers that are compatible with Java, .NET, JavaScript, Python, Ruby, etc. In this article, let's look at writing some Java code to do the CRUD (create, read, update and delete) operations. The prerequisites are Java[®] and the `jdbc-neo4j` driver. After downloading the JAR, create a new project in Eclipse and add the JAR to the project build path. Now create two files *DBConnectionManager.java* and *NodeManipulation.java* as shown in Figure 3.

The DBConnectionManager contains the code for connecting to the *neo4j* database where the URL, username

[illegible]

Figure 4: Add update